

CSA0607-Design and Analysis of Algorithms

CO2-AT2

Student Name: A. Sai Joshitha (192424220)

Q11. A banking system stores daily transaction amounts in a list where new entries are appended randomly, causing disorder. The system needs to sort data periodically for reporting.

- a) Analyze how Selection Sort and Bubble Sort would work in this scenario.**
- b) Compare the number of comparisons and swaps required by both methods.**
- c) Recommend the most suitable sorting technique with proper justification.**

Aim

To analyze the performance of Selection Sort and Bubble Sort for sorting daily banking transaction amounts.

a) Explanation

Selection Sort

- Finds the smallest element in the unsorted portion.
- Swaps it with the first unsorted element.
- Repeats until the list is sorted.

Bubble Sort

- Compares adjacent elements.
- Swaps them if they are in the wrong order.
- Repeats until no swaps occur.

Algorithm

Selection Sort

1. Start from first element.
2. Find minimum element.

3. Swap with current element.
4. Repeat.

Bubble Sort

1. Compare adjacent elements.
2. Swap if needed.
3. Repeat until sorted.

CODE

```
main.py
1 transactions = [5500, 1200, 7800, 3400, 2200]
2
3
4 selection = transactions.copy()
5
6 for i in range(len(selection)):
7     min_index = i
8     for j in range(i + 1, len(selection)):
9         if selection[j] < selection[min_index]:
10             min_index = j
11     selection[i], selection[min_index] = selection[min_index], selection[i]
12
13 print("Selection Sort:", selection)
14
15
16 bubble = transactions.copy()
17
18 for i in range(len(bubble)):
19     for j in range(len(bubble)-i-1):
20         if bubble[j] > bubble[j+1]:
21             bubble[j], bubble[j+1] = bubble[j+1], bubble[j]
22
23 print("Bubble Sort:", bubble)
```

Output:

```
Output
Selection Sort: [1200, 2200, 3400, 5500, 7800]
Bubble Sort: [1200, 2200, 3400, 5500, 7800]

=== Code Execution Successful ===
```

b) Comparison

Feature	Selection Sort	Bubble Sort
Comparisons	$O(n^2)$	$O(n^2)$
Swaps	$O(n)$	$O(n^2)$
Best Case	$O(n^2)$	$O(n)$
Worst Case	$O(n^2)$	$O(n^2)$

c) Recommendation

Selection Sort is preferable because it performs fewer swaps, making it more efficient when writing data is costly.

Q12. A delivery company maintains GPS coordinates of delivery points and computes distances between every pair to find the closest drop location. As the number of locations increases, delays are observed.

- a) Explain how the brute-force closest pair algorithm operates in this scenario.**
- b) Derive the time complexity of computing all pair distances.**
- c) Analyze why the performance degrades with large datasets.**

Aim

To find the minimum distance between delivery locations.

a) Explanation

The algorithm calculates the distance between every pair of delivery locations and stores the minimum distance.

Algorithm

1. Read all coordinates.
2. Compare every pair.
3. Calculate Euclidean distance.
4. Store minimum distance.

Code

```
import math

points = [(1,2),(4,6),(5,1),(8,3)]

min_distance = float('inf')

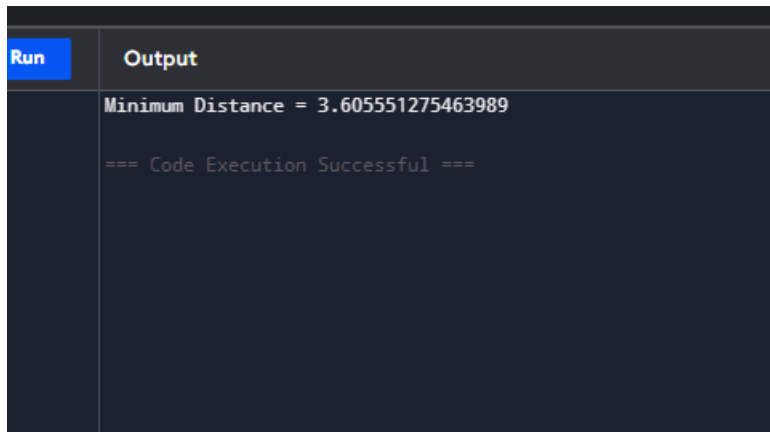
for i in range(len(points)):
    for j in range(i+1, len(points)):
        x1,y1 = points[i]
        x2,y2 = points[j]

        distance = math.sqrt((x2-x1)**2 + (y2-y1)**2)

        if distance < min_distance:
            min_distance = distance

print("Minimum Distance =", min_distance)
```

Output:



The screenshot shows a code execution interface with a dark background. On the left, there is a blue button labeled 'Run'. To its right, the word 'Output' is displayed in a light blue font. Below this, the output text is shown in a light blue monospace font. The first line of output is 'Minimum Distance = 3.605551275463989'. The second line is '=== Code Execution Successful ==='.

```
Run    Output
Minimum Distance = 3.605551275463989
=== Code Execution Successful ===
```

b) Time Complexity

Number of comparisons

$$n(n-1)/2$$

Time Complexity

$$O(n^2)$$

c) Why Slow?

- Every point is compared with every other point.
- Number of comparisons grows rapidly.
- Large datasets increase execution time.

Q13. A university portal stores student IDs in an unsorted format due to continuous registrations. During exams, quick lookup of student IDs is required.

a) Explain how Linear Search works for retrieving student records.

b) Analyze best, average, and worst-case time complexities.

c) Justify whether sorting the dataset would improve performance.

Aim

To search student IDs stored in an unsorted list.

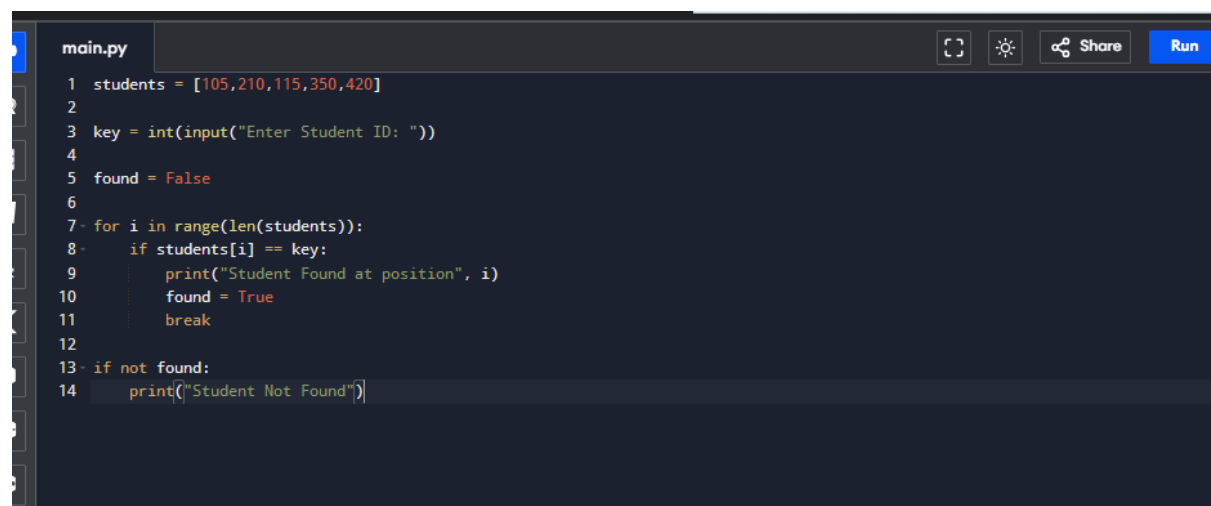
a) Explanation

Linear Search checks every student ID until the required ID is found.

Algorithm

1. Start from first element.
2. Compare with target.
3. If found, stop.
4. Otherwise continue.

Code

A screenshot of a code editor window titled 'main.py'. The editor has a dark background with light-colored text. The code is a Python program for linear search. It starts with a list of student IDs: [105, 210, 115, 350, 420]. It prompts the user to enter a student ID. It then iterates through the list, comparing each element with the input key. If a match is found, it prints the position and breaks the loop. If no match is found after the loop, it prints 'Student Not Found'. The code is as follows:

```
1 students = [105,210,115,350,420]
2
3 key = int(input("Enter Student ID: "))
4
5 found = False
6
7 for i in range(len(students)):
8     if students[i] == key:
9         print("Student Found at position", i)
10        found = True
11        break
12
13 if not found:
14     print("Student Not Found")
```

Output

```
Output
Student Found at Position: 3

=== Code Execution Successful ===
```

b) Time Complexity

Case	Complexity
Best	$O(1)$
Average	$O(n)$
Worst	$O(n)$

c) Justification

Sorting the data allows Binary Search.

Binary Search

$O(\log n)$

which is much faster for repeated searches.

Q14. A mapping application determines the outer boundary of a set of plotted land coordinates using a brute-force convex hull method. The dataset size increases significantly over time.

- a) Explain how the brute-force convex hull algorithm works in this case.**
- b) Analyze the computational complexity of checking all point pairs.**
- c) Discuss why this approach becomes inefficient for large inputs.**

Aim

To determine the outer boundary of plotted land coordinates.

a) Explanation

Every pair of points is checked.

If all remaining points lie on one side of the line, the pair belongs to the convex hull.

Code

```
main.py    Share
1 points = [(0,0),(2,0),(2,2),(0,2),(1,1)]
2
3 n = len(points)
4
5 print("Checking all point pairs")
6
7 for i in range(n):
8     for j in range(i+1,n):
9         print(points[i], "-", points[j])
```

Output

```
Output
Checking all point pairs
(0, 0) - (2, 0)
(0, 0) - (2, 2)
(0, 0) - (0, 2)
(0, 0) - (1, 1)
(2, 0) - (2, 2)
(2, 0) - (0, 2)
(2, 0) - (1, 1)
(2, 2) - (0, 2)
(2, 2) - (1, 1)
(0, 2) - (1, 1)

=== Code Execution Successful ===
```

b) Complexity

Checking every pair

$O(n^2)$

Checking all other points

$O(n)$

Overall

$O(n^3)$

c) Why Inefficient?

- Every pair is tested.
- Every remaining point must be checked.
- Large datasets produce very high computation time.

Q15. A plagiarism detection system scans large documents to match a given pattern using brute-force string matching. The document size is very large and pattern occurrences are rare.

- a) Explain how brute-force string matching operates in this scenario.**
- b) Analyze its time complexity.**
- c) Discuss why the approach becomes inefficient for large text inputs.**

Aim

To detect plagiarism by matching a pattern inside a document.

a) Explanation

The pattern is compared character by character with every possible position in the text.

Algorithm

1. Start from first character.
2. Compare pattern.
3. If mismatch occurs, shift by one position.
4. Repeat.

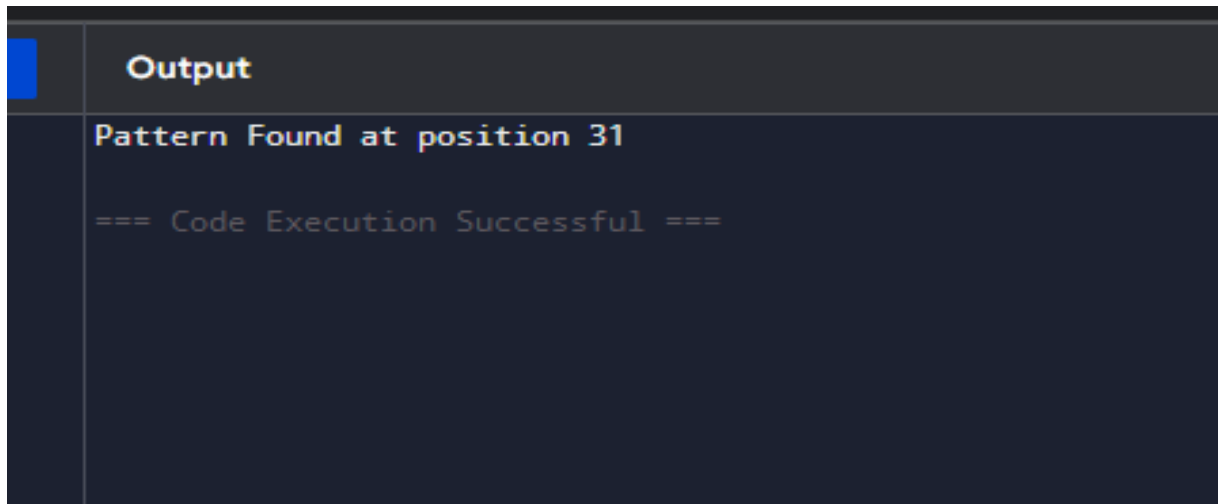
Code



```
Programiz
Python Online Compiler

main.py
1 text = "THIS IS A PLAGIARISM DETECTION SYSTEM"
2
3 pattern = "SYSTEM"
4
5 found = False
6
7 for i in range(len(text)-len(pattern)+1):
8
9     if text[i:i+len(pattern)] == pattern:
10         print("Pattern Found at position", i)
11         found = True
12
13 if not found:
14     print("Pattern Not Found")
```

Output

A screenshot of a terminal window with a dark background. The title bar at the top is dark gray with the word "Output" in yellow. The terminal content shows "Pattern Found at position 31" in a light blue font, followed by "=== Code Execution Successful ===" in a light gray font on the next line.

```
Output
Pattern Found at position 31
=== Code Execution Successful ===
```

b) Time Complexity

- Best Case: $O(n)$
- Worst Case: $O(n \times m)$

where:

- n = length of the text
- m = length of the pattern

c) Why Inefficient?

- The pattern is checked at every position in the document.
- Many unnecessary character comparisons are performed.
- For large documents with rare pattern occurrences, execution time increases significantly.